

vesuvius-challenge-surface-detection

8th-place-solution

Rank	8
Team	lingyundev
Author	lingyundev
Topic ID	679248
Version	Original

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679248>

Retrieved with Kaggle CLI on 2026-07-07.

Thanks to the organizers for hosting this technically challenging and topology-sensitive competition, and congratulations to all prize-winning teams.

One-line summary:

Multi-scale nnUNetResEncUNetL ensemble + mean fusion + probability diffusion + direction-aware adaptive hysteresis.

Models.

The backbone is nnUNetResEncUNetL, without architectural modification. Three models were trained with different patch sizes, focusing on MedialSurfaceRecall: patch 224 (500 epochs), patch 256 (500 epochs), and patch 288 (500 epochs). Different patch sizes provide complementary receptive fields, balancing local surface precision and global continuity.

Inference & fusion.

Inference followed the standard nnUNet sliding-window pipeline ($\text{step_size} = 0.9$). Patch 224 and 256 used mirroring TTA, while patch 288 did not due to inference speed constraints. The final prediction was obtained by direct mean fusion of the three probability maps, without logit weighting, which showed more stable leaderboard behavior.

Post-processing (key part).

Most performance gains came from post-processing. First, gradient anisotropic diffusion was applied twice to the fused probability map ($\text{conductance} = 2.0$, $\text{time_step} = 0.04$) to smooth fragmented responses while preserving sharp surface transitions in probability space. Second, adaptive hysteresis thresholding with directional morphology was applied. Parameters were $T_{\text{low}} = 0.50$, $T_{\text{high}} = 0.90$, $z_{\text{radius}} = 1$, $xy_{\text{radius}} = 0$, and $\text{min_size} = 100$. Voxels above T_{high} define high-confidence surface cores, while voxels between T_{low} and T_{high} are included only if connected to the core. Morphological connectivity is restricted to the z direction to reflect the thin layered structure of the scroll surface and prevent lateral over-expansion, followed by removal of small isolated components.

A lighter diffusion variant (single iteration, $\text{conductance} = 1.5$, $\text{time_step} = 0.03$) slightly improved private performance (up to 0.618) but reduced the public score ($0.595 \rightarrow 0.593$). Compared to using adaptive hysteresis alone, diffusion improved local validation by ~ 0.005 , although its public score was slightly worse. We therefore selected the stronger diffusion setting, trusting the local evaluation and prioritizing overall stability.

Overall, the solution avoids complex architectural tricks and relies on scale diversity, stable mean ensembling, probability-space smoothing, and direction-aware adaptive thresholding.

Due to the Chinese New Year holiday, I was not able to devote much time to the competition during the final two weeks; given that, I am very satisfied with the final ranking. Thanks again to the organizers for this unique and inspiring challenge.

The inference notebook is [here](#). Code is [here](#).

Supplemental Q&A

CPMP · 2026-02-28T12:30:26.530000

Interesting. I didn't know about "gradient anisotropic diffusion".

Is it different from anisotropic diffusion as in wikipedia https://en.wikipedia.org/wiki/Anisotropic_diffusion ?

You made me search a bit, and I saw on wikipedia that anisotropic diffusion is a generalization of Gaussian smoothing. I used the latter, I wish I knew about the more general form.

Further search makes em wonder about which implementation you used. Found some C++ ones, and this python one <https://github.com/SimpleITK/SlicerSimpleFilters/blob/master/SimpleFilters/Resources/json/GradientAnisotropicDiffusionImageFilter.json>

lingyundev · 2026-02-28T14:07:15.790000

Yes, it is closely related to the anisotropic diffusion described on Wikipedia (Perona–Malik). By “gradient anisotropic diffusion”, I simply mean that the diffusion strength is controlled by the local gradient magnitude, so smoothing is suppressed across strong edges and encouraged in flat regions, unlike Gaussian smoothing.

The main difference in my usage is that diffusion is applied to the model output probability map, not to the raw image. This helps smooth fragmented responses while preserving thin surface boundaries. The implementation is equivalent in behavior to ITK / SimpleITK's GradientAnisotropicDiffusionImageFilter.